

Problem 1

Wittrup problem 8-6. Please simplify the problem by assuming that the total number of bacterial drug targets in the bloodstream is much less than the number of your drug in the bloodstream (in other words, you don't need to account for drug binding and dissociation in your drug mass balance). Also simplify the problem by assuming an average of 2.3 drug-target complexes per bacterium (in part b) and binding is monovalent; the drug clearance rate constant is 1.2 h^{-1} , the plasma volume is 4.0 L, and the monovalent dissociation constant for the drug and bacterial drug target is 4 nM.

An enterprising young faculty member sets out to measure the number of receptors of a certain type on a given cell line. Her method relies on using diffusion-limited characteristics of ligand binding to reveal receptor number. She has prepared specific cross-linking reagents that bind covalently to a portion of the receptor away from the ligand binding site. With these cross-linkers, she will make separate preparations. One will contain only single cells, another will contain cross-linked pairs of cells, yet another will contain cross-linked triples, and so on for higher-order oligomers. Assume that the equations derived for spherical single cells also apply to the oligomers created. Using cell-sorting technology, she will be able to get exact counts of numbers of cells.

Problem 1a

What experiments will this young faculty member carry out to measure the number of receptors per cell?

Measure diffusion-limited on-rates for singlets, doublets, triplets, etc., plot k_{on}^{obs} vs. number of cells per oligomer which produces a linear, and the slope gives you N (receptors/cell) via $k_{on}^{diff} = 4\pi DNa$

Problem 1b

What other measurements will she need to obtain?

From (a), the slope of k_{on}^{obs} vs. m (cells per oligomer) $= 4\pi DNa$, so to solve for N :

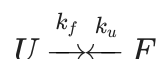
$$N = \frac{\text{slope}}{4\pi Da}$$

She additionally needs:

1. D — diffusion coefficient of the ligand in solution (e.g., via DLS or Stokes–Einstein)
2. a — cell radius (e.g., via microscopy or flow cytometry)

Problem 2

Poisson processes. Single molecule detection is commonly used in this building to estimate kinetic rate processes. As one example, the Kaar and Schwartz groups (ref 1) labeled a protein with two fluorophores, and used FRET efficiency to monitor the transition between a folded state (F) and an unfolded state (U) assuming a two-state folding model.



They determined the kinetic rate of unfolding, k_u , by counting the number of times their protein was observed in a folded state and stayed in a folded state (N_{ff}), and the number of times their protein was in a folded state and then unfolded in the next time period (N_{fu}). The equation correlating the kinetic rate of unfolding to these experimental observables are given by:

$$k_u = -\frac{1}{\tau} \log \left(\frac{N_{ff}}{N_{ff} + N_{fu}} \right)$$

Consider the N_{fu} to be governed by a Poisson process. τ is the time period between images, and is 80 ms (milliseconds) under the conditions of the paper.

Problem 2a

Consider a case where $N_{fu} = 8$, and $N_{ff} = 1700$. Determine the most likely unfolding rate in units of inverse seconds.

$$k_u = -\frac{1}{0.080} \ln \left(\frac{1700}{1700+8} \right) = -12.5 \cdot \ln \left(\frac{1700}{1708} \right)$$

$$\implies -12.5 \cdot \ln(0.99532) = -12.5 \cdot (-0.004691)$$

$$\therefore \boxed{k_u \approx 0.0587 \text{ s}^{-1}}$$

Problem 2b

Determine the probability that this unfolding rate is the true rate. In other words, what is the probability that $N_{fu} = 8$ events were found during the measurement period if the mean number of expected events were 8.

Poisson PMF with $\lambda = 8, k = 8$:

$$P(X = 8) = \frac{e^{-8} \cdot 8^8}{8!} = \frac{0.000335 \cdot 16777216}{40320}$$

$$\therefore \boxed{P \approx 0.1396}$$

Problem 2c

Determine upper and lower bounds of the most likely unfolding rate with 95% confidence. One way of estimating this confidence is finding the range of number of expected events within a probability of 95% given that 8 events were observed. In other words, find the lower bound (lb) and the upper bound (ub) of expected events able to satisfy this equation

$$\sum_{\mu=lb}^{\mu b} P(8|\mu) \geq 0.95$$

$$\lambda_{lb} = 3.454, \quad \lambda_{ub} = 15.763$$

Plugging into $k_u = -\frac{1}{\tau} \ln\left(\frac{N_{ff}}{N_{ff} + \lambda}\right)$ with $N_{ff} = 1700$:

$$k_u^{lb} = -12.5 \ln\left(\frac{1700}{1703.454}\right) \approx 0.0253 \text{ s}^{-1}$$

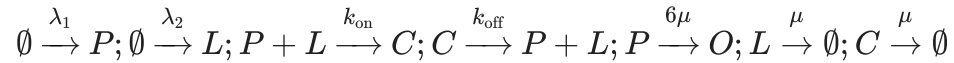
$$k_u^{ub} = -12.5 \ln\left(\frac{1700}{1715.763}\right) \approx 0.1147 \text{ s}^{-1}$$

$$\therefore \boxed{k_u \in [0.0253, 0.1147] \text{ s}^{-1}}$$

Problem 3

Chemical master equation. Write the chemical master equation for the following biochemical network governed by these rate equation. This family of equations should be a time derivative of $P_{m,n,c}$, where P is the probability, m is the # of proteins P, n is

the # of ligands L, and r is the # of complexes C. (1 point for each term written correctly, 1 point for overall master equation).



$$\frac{dP_{m,n,c}}{dt} =$$

$$\lambda_1(P_{m-1,n,c} - P_{m,n,c})$$

$$+ \lambda_2(P_{m,n-1,c} - P_{m,n,c})$$

$$+ k_{\text{on}}[(m+1)(n+1)P_{m+1,n+1,c-1} - mnP_{m,n,c}]$$

$$+ k_{\text{off}}[(c+1)P_{m-1,n-1,c+1} - cP_{m,n,c}]$$

$$+ 6\mu[(m+1)P_{m+1,n,c} - mP_{m,n,c}]$$

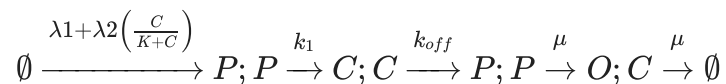
$$+ \mu[(n+1)P_{m,n+1,c} - nP_{m,n,c}]$$

$$+ \mu[(c+1)P_{m,n,c+1} - cP_{m,n,c}]$$

Problem 4

Stochastic simulation. Cell fate decisions by mammalian cells in response to viral infection can be stochastic – only 20% of cells will produce interferon beta. Another way of saying this is that every cell can produce interferon beta, given enough time, but at a given time point only a subset would be able to do so. Literature (ref 2) suggests that stochasticity is governed by low mRNA copy numbers encoding proteins which sense viral DNA, proteins which activate the signaling pathway, to the transcription factors which promote expression of interferon beta. Let's consider a simplified version of this pathway, which operates by positive feedback. Consider a protein P which senses viral DNA. Assume for simplicity that the amount of DNA is constant and in great excess over the number of proteins. The protein can interact with DNA, forming a complex C. The complex C can stimulate production of the protein P. Production of interferon beta occurs when the number of complexes reach 10 or more. Degradation of P and C occur using the same rate constant μ and are 1st order. There are many simplifications to the system – we're not considering mRNA transcription and translation, we're not considering other proteins

in the pathway, we're not considering the gene regulatory network for interferon beta gene expression, and so forth. The biochemical network is shown below.



Problem 4a

Write the stoichiometric matrix and the associated rate vector for this process.

$$\mathbf{x} = [P, C]^T$$

Stoichiometric matrix: (rows = species, cols = reactions):

$$S = \begin{bmatrix} +1 & -1 & +1 & -1 & 0 \\ 0 & +1 & -1 & 0 & -1 \end{bmatrix}$$

Rate vector:

$$\mathbf{r} = \begin{bmatrix} \lambda_1 + \lambda_2 \frac{C}{K+C} \\ k_1 P \\ k_{off} C \\ \mu P \\ \mu C \end{bmatrix}$$

Problem 4b

Simulate this stochastic network using initial amounts of P, C = 0. Show a plot of number of complexes C vs. time (in hours). Determine the amount of time it takes to reach 10 complexes.

```
In [7]: import numpy as np
import matplotlib.pyplot as plt
```

```

# Parameters (units: h-1)
lambda1 = 0.2
lambda2 = 20.0
k1       = 20.0
koff     = 10.0
mu       = 1.0
K        = 1.0 # half-saturation for feedback (molecules)

def gillespie(t_end=30.0, seed=None):
    rng = np.random.default_rng(seed)
    P, C = 0, 0
    t = 0.0
    ts, Cs = [0.0], [0]
    t_hit = None

    while t < t_end:
        r = np.array([
            lambda1 + lambda2 * C / (K + C), # production of P
            k1 * P,                          # P -> C
            koff * C,                        # C -> P
            mu * P,                          # P degradation
            mu * C,                          # C degradation
        ])
        R = r.sum()
        if R == 0:
            break

        dt = rng.exponential(1.0 / R)
        t += dt

        u = rng.uniform(0, R)
        cumr = np.cumsum(r)
        if u < cumr[0]:
            P += 1
        elif u < cumr[1]:
            P -= 1; C += 1
        elif u < cumr[2]:
            C -= 1; P += 1
        elif u < cumr[3]:
            P -= 1
        else:

```

```

        C -= 1

    P = max(P, 0)
    C = max(C, 0)
    ts.append(t)
    Cs.append(C)

    if t_hit is None and C >= 10:
        t_hit = t

    return np.array(ts), np.array(Cs), t_hit

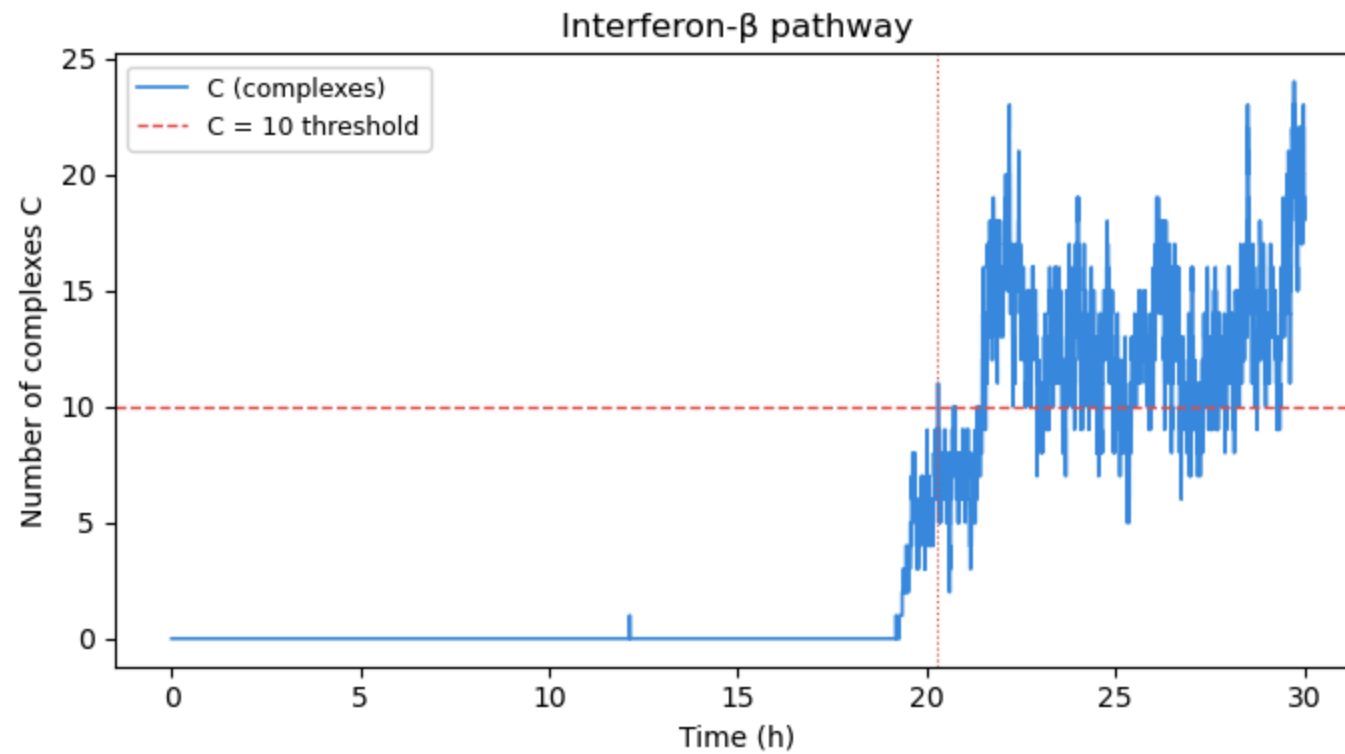
if __name__ == "__main__":
    ts, Cs, t_hit = gillespie(seed=42)

    print(f"Time to reach C >= 10: {t_hit:.4f} h" if t_hit else "C = 10 not reached")

    fig, ax = plt.subplots(figsize=(7, 4))
    ax.step(ts, Cs, where="post", color="#378ADD", linewidth=1.2, label="C (complexes)")
    ax.axhline(10, color="#E24B4A", linestyle="--", linewidth=1.0, label="C = 10 threshold")
    if t_hit:
        ax.axvline(t_hit, color="#E24B4A", linestyle=":", linewidth=0.8)
        # ax.annotate(f"t = {t_hit:.2f} h", xy=(t_hit, 10),
        #             xytext=(t_hit + 0.3, 11), fontsize=9, color="#E24B4A")
    ax.set_xlabel("Time (h)")
    ax.set_ylabel("Number of complexes C")
    ax.set_title("Interferon- $\beta$  pathway")
    ax.legend(fontsize=9)
    plt.tight_layout()
    plt.show()

```

Time to reach C >= 10: 20.2980 h



Problem 4c

Repeat the simulation in part b 50 times. Plot the distribution of the amount of time it takes to reach 10 complexes. Determine the time in hours for 20% of the cells to be activated.

Parameter	Value	Units
λ_1	0.2	h^{-1}
λ_2	20	h^{-1}
k_1	20	h^{-1}
k_{off}	10	h^{-1}
$\mu.$	1.	h^{-1}

```
In [6]: import numpy as np
import matplotlib.pyplot as plt

lambda1 = 0.2
lambda2 = 20.0
k1       = 20.0
koff     = 10.0
mu       = 1.0
K        = 1.0

def gillespie(t_end=30.0, seed=None):
    rng = np.random.default_rng(seed)
    P, C = 0, 0
    t = 0.0
    ts, Cs = [0.0], [0]
    t_hit = None
    while t < t_end:
        r = np.array([
            lambda1 + lambda2 * C / (K + C),
            k1 * P,
            koff * C,
            mu * P,
            mu * C,
        ])
        R = r.sum()
        if R == 0:
            break
        dt = rng.exponential(1.0 / R)
        t += dt
        u = rng.uniform(0, R)
        cumr = np.cumsum(r)
        if u < cumr[0]:
            P += 1
        elif u < cumr[1]:
            P -= 1; C += 1
        elif u < cumr[2]:
            C -= 1; P += 1
        elif u < cumr[3]:
            P -= 1
        else:
            C -= 1
```

```

    P = max(P, 0)
    C = max(C, 0)
    ts.append(t)
    Cs.append(C)
    if t_hit is None and C >= 10:
        t_hit = t
    return np.array(ts), np.array(Cs), t_hit

N = 50
hit_times = []
for i in range(N):
    _, _, t_hit = gillespie(t_end=30.0, seed=i)
    if t_hit is not None:
        hit_times.append(t_hit)

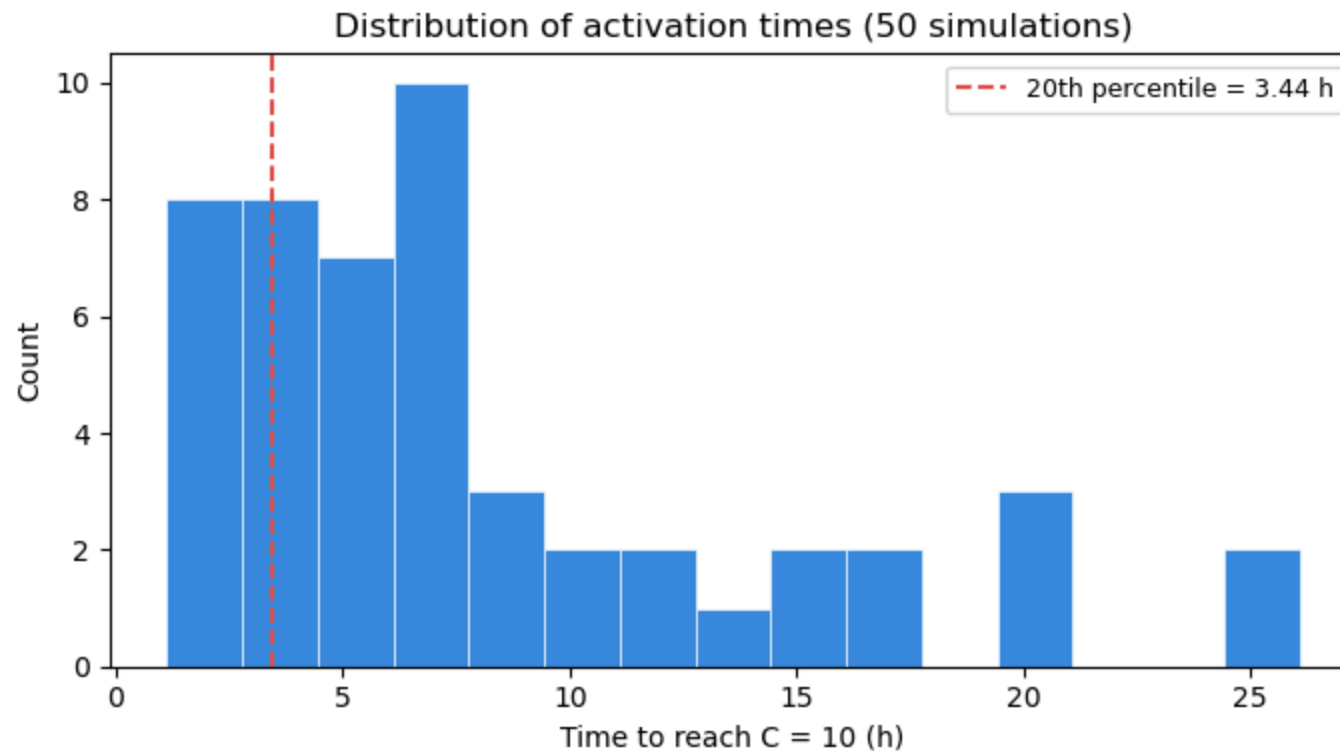
hit_times = np.array(hit_times)
t20 = np.percentile(hit_times, 20)

print(f"Runs that reached C>=10: {len(hit_times)}/{N}")
print(f"20th percentile activation time: {t20:.4f} h")

fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(hit_times, bins=15, color="#378ADD", edgecolor="white", linewidth=0.5)
ax.axvline(t20, color="#E24B4A", linestyle="--", linewidth=1.5,
           label=f"20th percentile = {t20:.2f} h")
ax.set_xlabel("Time to reach C = 10 (h)")
ax.set_ylabel("Count")
ax.set_title("Distribution of activation times (50 simulations)")
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

```

Runs that reached C>=10: 50/50
 20th percentile activation time: 3.4369 h



Location of Jupyter Notebook

This file was created in jupyter notebook, rendered into `.html` through `jupyter nbconvert --to html` PS12/PS12.ipynb then the `.html` file was saved as a `.pdf`. The location of all said files are github.com/caterer-zt/CHEN_5150/PS12

Please contact ztcaterer@colorado.edu if there are any issues or concerns.

Thanks :-)

Disclosure

Generative artificial intelligence tools were used to assist with this project. All generated content was reviewed, edited, and verified by the authors, who take full responsibility for the accuracy, integrity, and final content of this work.

